

锐能微 SOC_MCU 应用笔记 003- 库函数使用说明

Rev3.0

修改记录

版本号	拟制/修改日期	更改理由	主要更改内容
V1.0	2013.1	创建	初始版本
V1.1	2013.7	更正	删除不再使用的函数
V1.2	2017.3	增加功能，支持新的产品型号（RN8213（C版），RN8211B（C），RN8318，RN861x），并向下兼容原有SOC/MCU系列产品	1.系统模式切换函数中增加 3.2MHz RC 和 14.7456MHz PLL 模式 2.增加 SysUpdate()函数 3.NVM 库函数更新，支持新增产品（RN8213（C版），RN8211B（C版），RN8318，RN861x）
V1.3	2017.8	完善功能	1、 RtcWriteDota 增加返回值 2、 增加 Dota 的含义及计算公式说明 3、 删除 eepromPageErase 及 eepromSectorErase 函数及其说明
V2.0	2023-3-24	RN8318/RN8613/RN8615 等 MCU 产品从 V1 版本升级到 V2 版本后，库函数也做了相应升级，因此对应用笔记做出修订； 锐能微 V2 版本 MCU 产品包括以下型号： RN8318、RN8613、RN8615（V1 及 V2 都有的型号） 新增型号：RN8611	1、新增系统参数校验库函数 SysConfCheck ，要求用户必须定时调用该函数去校验芯片内部的系统参数，定时周期可以选择 1 秒到 1 分钟之间。 2、SysModeChg 新增 PLL29M 和 RC29M 两种新模式 3、新增库函数版本获取函数，客户可通过库函数版本获取函数得到库函数的版本，以追溯库函数版本。 4、代码优化：系统模式切换、REMAP4 选择库函数、NVM 库函数、及 RTC 校准这类原有库函数的返回类型修改成枚举类型；使用新的库函数后，接收库函数返回值的变量需做修改。 5、文件合并：三个库函数合并成 1 个库函数，具体说明如下：在 keil 应用环境，以前需要用到 nvm.lib nvm.h/sysctrl.lib/sysctrl.h/rtc.lib/rtc.h 这 6 个文件，现在只需用 rn8xxx_v2_lib.lib/rn8xxx_v2_lib.h 这两个文件；在 IAR 应用环境，以前需要用到 nvm.a/nvm.h 等 6 个文件，现在只需用 rn8xxx_v2_lib.a/rn8xxx_v2_lib.h 这两个文件。两种开发环境下，都是将原来的 6 个文件替换成现在发布的 2 个文件即可；

V2.1	2023-4-13	新增 RN8xxx MCU 芯片 V2 版本的 FLASH 参数说明	1、3.2 函数说明章节增加 RN8xxx MCU 芯片 V2 版本的 FLASH 参数说明
V2.2	2023-6-14	库函数升级	增加 SysConfCheck 接口函数调用重点注意事项
V2.3	2023-10-25	库函数升级	1、增加快速时钟切换函数接口 FastSysModeChg 2、增加用于 V2 版本芯片的时钟模式 Clock_PLL_58M9_DIV2
V2.4	2023-11-16	增加函数接口	增加简化版系统参数校验函数接口 FlashVoltageTrim
V2.5	2023-12-12	增加函数接口	1、增加 FLASH 编程函数参数的说明 2、增加选项字节操作库函数接口说明
Rev2.6	2024-5-31	库函数更新到 1.2.1: 三相 SOC 新增 V2 版本 MCU 和单相 SOC 新增 V3 版本 当前库函数体量大、运行速度慢,需缩减代码,提高运行速度	1、增加精简版库函数,不同产品型号提供了不同编译选项的库函数,这样调用精简版库函数时,可减少库函数代码量,降低库函数的运行时间;具体情况见“库函数 V1.2.0 用户更新指引”。 2、增加对三相 SOC V2 版本芯片的支持 3、增加对 MCU V3 和 SOC V3 版本芯片的支持 4、增加 RCH 时钟调整函数接口,见第 9 章 5、增加合封 EEPROM 操作函数接口,见第 8 章 6、修复 MISRA 和 PC-LINK 扫描出的警告
Rev2.7	2024-8-2	库函数更新到 1.2.1: 增加时钟类库函数接口	1、增加内容:库函数更新到 1.2.1,增加时钟类库函数接口,及应用方法和应用注意事项
Rev2.8	2024-8-19	库函数更新到 1.2.2: 非 II 版本与 II 版本库函数合并	1、重大变更:库函数更新到 1.2.2,使用 II 版本头文件重新封装非 II 版本接口,删除非 II 版本源码,源码统一为 II 版本。非 II 版本应用程序接口保持不变,工程源文件需要重新添加。同时增加函数接口 2、增加描述:flashProgram 函数增加源地址字对齐时,长度要求为页大小的整数倍或小于页字节数;源地址非字对齐时,无编程长度限制,但函数内做数据搬移,有性能损耗;推荐使用源地址字对齐方式。
Rev2.9	2024-8-30		1、增加 MCU V3 内嵌库函数使用方法说明

Rev2.10	2025-3-17		1、增加了 RtcWriteDota 函数，dota0 寄存器的计算方法。 2、时钟切换函数 SysModeChg，针对三相 soc 的典型频率为 32.768Mhz，增加描述。
Rev3.0	2025-4-25	内置库函数支持范围扩大,库函数版本升级到 V1.2.6	1、内置库函数支持范围扩大到单相 SOC V2/V3 版、三相 SOC V2 版及 MCU V2/V3 版，不再仅限于 MCU V3 版芯片； 2、修订内置库函数使用方法，包括需要在 renergy_chip_config.h 文件内选择芯片系列号以及开启 NVRLIB_EN 宏，并增加源文件 rn8xxx_ll_lib.c； 3、增加库函数 V1.2.6 用户更新指引； 4、库函数接口名称与源码函数名统一，与之前的差异是在原来函数名前增加 LL_ 文件名前缀； 5、增加 RTC\WDT\RCH\RCL 芯片参数校验库函数接口，LL_SYSOPTION_RtcConfCheck 等，以满足有客户需要自行管理相关参数的需求。现有系统配置参数校验库函数 LL_SYSOPTION_SysConfCheck，不包含对 RTC\WDT\RCH\RCL 参数的校验，若用户需要对上述模块校验，可调用相关新增函数。 6、增加时钟校正函数应用说明； 7、按照发布库函数源码结构，调整库函数应用笔记结构，如 NVM 库函数章节拆分成 FLASH 操作库函数和 EEPROM 操作库函数。

重点注意事项

1、LL_SYSOPTION_SysConfCheck 接口函数调用注意事项：

1.1 复位后，必须先调用该函数，才能进行其他操作；

1.2 程序正常运行时要求用户必须定时调用该函数去校验芯片内部的系统参数，定时周期可以选择 1 秒到 1 分钟之间。休眠后，建议定时调用该函数，为平衡功耗，定时周期可选择 10 分钟以上。

1.3 当客户有 BOOT 程序时，BOOT 程序也要执行上述 1、2 两步操作，如代码空间不够，可改为调用 LL_SYSOPTION_FlashVoltageTrim 函数。

2、LL_SYSOPTION_FlashVoltageTrim 接口函数调用注意事项：

该函数为 LL_SYSOPTION_SysConfCheck 的简化版函数。在 V2 芯片进行 FLASH 擦写前至少需要调用一次该函数。

3、库函数更新到 V1.2.2 时，合并了 II 形式和非 II 形式的库函数，两种形式库函数的函数名称不同，II 形式的函数名称前带有相应文件的前缀，非 II 形式的函数名前没有前缀，合并后两者等效，故本库函数应用笔记不再分别叙述，只要最后的名称一致即为等效函数。

目录

修改记录	1
重点注意事项	4
目录	5
库函数 V1.0.0 用户更新指引	8
库函数 V1.2.0 用户更新指引	10
库函数 V1.2.2 用户更新指引	12
库函数 V1.2.6 用户更新指引	14
芯片内置库函数使用方法说明	15
1 系统模式切换库函数	16
1.1 函数接口	16
1.2 函数说明	16
时钟切换库函数	16
快速时钟切换函数	18
全失压激活函数	19
全失压休眠函数	19
2 库函数版本获取函数	19
2.1 函数接口	19
2.2 函数说明	19
库函数版本获取函数	19
库函数源码版本获取函数	20
3 FLASH 操作库函数	20
3.1 函数接口	20
3.2 函数说明	20
FLASH 页擦除函数	22
FLASH 块擦除函数	22
FLASH 编程函数	23
cache 打开函数	23
cache 关闭函数	24
4 EEPROM 操作库函数	24
4.1 函数接口	24
4.2 函数说明	24
EEPROM 编程函数	25
EEPROM 进入低功耗函数	26
EEPROM 唤醒函数	26
5 RTC 操作库函数	26
5.1 函数接口	26
5.2 函数说明	26
RTC DOTA 参数读函数	26
RTC DOTA 参数写函数	27

RTC 时钟源选择函数	27
6 芯片参数校验库函数	28
6.1 函数接口	28
6.2 函数说明	28
系统配置参数校验库函数	28
RTC 配置参数校验库函数	29
WDT 配置参数校验库函数	29
RCH 校正参数校验库函数	30
RCL 校正参数校验库函数	30
FlashVoltageTrim 系统参数配置库函数	31
7 选项字节操作函数	31
7.1 函数接口	31
7.2 函数说明	32
Dota0 配置函数	32
Alphal 配置函数	32
Alphah 配置函数	33
Xt0 配置函数	33
Remap 配置函数	33
保护等级配置函数	34
Dota0 获取函数	34
Alphal 获取函数	35
Alphah 获取函数	35
Xt0 获取函数	36
Remap 获取函数	36
保护等级获取函数	36
MCU 版本获取函数	37
8 合封 EEPROM 操作函数	37
8.1 函数接口	37
8.2 函数说明	37
合封 EERPOM 数据读取函数	37
合封 EERPOM 写函数	38
合封 EERPOM 初始化	38
合封 EERPOM 复位函数	39
9 时钟调整函数	39
9.1 函数接口	39
9.2 函数说明	39
RCH 校准函数	39
RCL 校准函数	40
系统时钟同步监控函数	41
系统时钟异步监控函数	41
系统时钟异步监控结果获取函数	42
LOSC 停振检测函数	42
10 GPADC 操作函数	43
10.1 函数接口	43

10.2 函数说明	43
GPADC 温度偏置获取函数	43
11 计量操作函数	44
11.1 函数接口	44
11.2 函数说明	44
外部基准控制函数	44
12 内置库校验函数	45
12.1 函数接口	45
12.2 函数说明	45
内置库校验函数	45

库函数 V1.0.0 用户更新指引

本用户更新指引仅适用于本文档“修订记录 V2.0”所述更改内容。

下表为库函数 V1.0.0 在 V1 芯片库函数上的具体修订内容列表：（V1 芯片库函数没有版本返回函数，故统一称为 V1 芯片库函数，从 V1.0.0 开始，提供库函数版本号）

序号	修改点	具体修改内容
1	源文件增删	1、删除 nvm/rtc/sysctrl.a (KEIL) /.lib(IAR)库函数文件 2、删除 nvm/rtc/sysctrl.h 头文件 3、添加 rn8xxx_v2_lib.a(.lib)库函数文件 4、添加 rn8xxx_v2_lib.h 头文件
2	代码头文件引用修改	1、将所有引用了 nvm/rtc/sysctrl.h 头文件的源代码修改为引用 rn8xxx_v2_lib.h
3	函数返回类型修改	1、sysctrl 相关函数（SysModeChg、Remap4Sel、SysConfCheck）函数返回值修改为枚举类型 eSysCtrlRet_TypeDef（值内容无变化） 2、rtc 相关函数（RtcWriteDota）函数返回值修改为枚举类型 eRtcRet_TypeDef（值内容无变化） 3、nvm 相关函数（flashPageErase、flashSectorErase、flashProgram、eepromProgram）函数返回值修改为枚举类型 eNvmRet_TypeDef（值内容无变化）
4	新增接口	1.SysConfCheck：系统参数校验，需保持周期性（1 秒~1 分间）调用 2.GetLibVersion：获取库函数软件版本 3.GetCodeVersion：获取库函数代码版本

V1 芯片库函数切换到库函数 V1.0.0 及以上版本库函数的具体操作步骤如下：

1、工程文件替换：

1.1 IAR 应用环境：将原来工程内的 6 个文件 nvm.a/nvm.h /rtc.a/ rtc.h sysctrl.a/ sysctrl.h 替换成现在的 2 个文件 rn8xxx_v2_lib.a 和 rn8xxx_v2_lib.h。

1.2 KEIL 应用环境：将原来工程内的 6 个文件 nvm.lib/nvm.h /rtc.lib/ rtc.h/ sysctrl.lib/ sysctrl.h 替换成现在的 2 个文件 rn8xxx_v2_lib.lib 和 rn8xxx_v2_lib.h。

1.3 直接使用库函数源码的应用，不区分应用环境，统一替换相应名字的.c 文件，nvm.h、rtc.h、sysctrl.h 替换成 rn8xxx_v2_lib.h。

2、工程配置修改：

2.1 IAR 应用环境：工程配置中，删除原来的库函数 nvm.a、rtc.a、sysctrl.a，添加本次提供的库函数 rn8xxx_v2_lib.a。

2.2 KEIL 应用环境：工程配置中，删除原来的库函数 `nvm.a`、`rtc.a`、`sysctrl.a`，添加本次提供的库函数 `rn8xxx_v2_lib.lib`。

3、工程文件中的头文件包含调整：原来包含头文件 `nvm.h`、`rtc.h`、`sysctrl.h` 的地方全部替换成 `rn8xxx_v2_lib.h`。

4、在应用程序内增加定时调用系统参数校验库函数 `SysConfCheck`，定时调用周期可以选择 1 秒到 1 分钟之间。该步骤仅针对 V2 版本 MCU 产品 RN8318\RN861X，其他产品不需调用。

5、在应用程序内修改接收库函数返回值的变量类型为对应枚举类型。

6、在应用程序内增加版本获取函数 `GetLibVersion` 及 `GetCodeVersion` 调用，并增加获取该函数返回值的通讯支持。该功能只是用于追溯库函数版本，客户也可以不调用版本获取函数；也可只调用该函数，但不增加获取该函数返回值的通讯支持。

库函数 V1.2.0 用户更新指引

本用户更新指引仅适用于本文档“修订记录 Rev2.6”所述更改内容。

当用户从库函数 V1. x. x 升级到 V1.2.0 时，需按照本指引进行操作。

当用户从 V1 芯片库函数升级到 V1.2.0 时，需先按照“库函数 V1.0.0 用户更新指引”操作后，再按照该用户更新指引操作。

鉴于锐能微芯片增加三相 SOC V2 版、MCU V3 版和单相 SOC V3 版，锐能微库函数升级到 V1.2.0，增加了相关接口。另外，当前库函数适用于锐能微所以芯片，为平台化库函数，为缩减代码，提高运行速度，库函数 V1.2.0 做了精简版库函数的支持。

精简版库函数仅适用于对应型号产品。

库函数分别以库函数形式和源码形式提供。

1、库函数形式

目前提供支持 KEIL 和 IAR 的库函数，分别会提供两个文件，一个是库文件，一个头文件。IAR 库文件后缀为 .a，KEIL 库文件后缀为 .lib，头文件后缀为 .h；库函数文件名称一样。

下表为 V1.2.0 提供的适应各型号产品的库函数文件名称及适应型号列表。

鉴于名称相同，为了看起来清晰，下表库函数文件名称只列出了 IAR 库函数文件名。

锐能微各型号产品的版本的区分，可参考相应的芯片升级说明。

序号	库函数文件名称	适用型号	说明
1	rn8xxx_v2_lib.a	所有 SOC/MCU 产品	平台版库函数
2	RN831x_RN861x_MCU_V1.a	MCU V1 版 RN8318\RN8615\RN8613\RN8612	精简版库函数
3	RN831x_RN861x_MCU_V2.a	MCU V2 版 RN8318\RN8615\RN8613\RN8611	精简版库函数
4	RN831x_RN861x_MCU_V3.a	MCU V3 版 RN8613\RN8610	精简版库函数
5	RN821x_RN721x_SOC_B.a	单相 SOC B 版 RN8211\RN8213\ RN8213B \RN8215\RN7213	精简版库函数
6	RN821x_RN721x_SOC_C.a	单相 SOC C 版 RN8213\RN8211B\RN8211	精简版库函数
7	RN821x_RN721x_SOC_D.a	单相 SOC D 版 RN8217\RN8213B\RN7213	精简版库函数
8	RN821x_RN721x_SOC_V2.a	单相 SOC V2 版 RN8213B\RN8217\RN8215\RN7213\ RN8213\RN8211B	精简版库函数
9	RN821x_RN721x_SOC_V3.a	单相 SOC V3 版	精简版库函数

		RN8213\RN8211B	
10	RN202X_RN7326_SOC_B.a	三相 SOC B 版 RN2025\RN2026\RN7326	精简版库函数
11	RN202X_RN7326_SOC_V2.a	三相 SOC B 版 RN2026\RN7326\RN2028	精简版库函数

2、源码形式

锐能微库函数还提供源码，本次库函数源码，增加了 `clk_trim.c` 和 `sip_eeprom.c` 两个文件，具体参见下表：

文件类型	v1.2.0	v1.1.7
源文件	<code>eepromProgram.c</code> <code>nvm.c</code> <code>rtc.c</code> <code>sysctrl.c</code> <code>system_cfg_update.c</code> <code>sysupdate.c</code> <code>utils.c</code> <code>clk_trim.c</code> （新增） <code>sip_eeprom.c</code> （新增）	<code>eepromProgram.c</code> <code>nvm.c</code> <code>rtc.c</code> <code>sysctrl.c</code> <code>system_cfg_update.c</code> <code>sysupdate.c</code> <code>utils.c</code>
头文件	<code>rn8xxx_v2_lib.h</code> <code>sysctrl_reg.h</code> <code>utils.h</code>	<code>rn8xxx_v2_lib.h</code> <code>sysctrl_reg.h</code> <code>utils.h</code>

建议客户使用库函数源码，这样即可做到平台化，又可享受精简版库函数的代码量少和运行速度快的优点。实现方法是，在 IDE 工具上全局定义所需产品型号对应的宏定义，其所需型号的宏定义名称即精简版库函数的名称，可参见上一章节“1、库函数形式”中表格“库函数文件名称”列中内容，如 RN8217 V2 版宏定义名称为 `RN821x_RN721x_SOC_V2`；如不做定义，则默认为平台版库函数。

库函数 V1.2.2 用户更新指引

本用户更新指引仅适用于本文档“修订记录 Rev2.8”所述更改内容。

从库函数 V1.x.x 升级到 V1.2.2 时，需按照本指引进行操作。

当用户从 V1 芯片库函数升级到 V1.2.2 时，需先按照“库函数 V1.0.0 用户更新指引”操作后，再按照该用户更新指引操作。

本更新指引适用于使用库函数源码的应用，如果库函数使用的是库，如 keil 使用 .lib，IAR 使用 .a 文件，库文件可直接替换，同时需将 inc 目录下的头文件全部添加到工程目录下，无需修改源代码和工程文件。

本次库函数的重要改动是合并非 II 和 II 形式库函数；其中非 II 形式库函数指早期提供的库函数；II 形式库函数指，2023 年开始提供的库函数，根据硬件功能模块进行文件划分，库函数文件名带“II_模块名_”标识，源文件.c 与头文件.h 保持一一对应关系。

II 形式库函数的函数名前带“II_模块名_”前缀，非 II 形式库函数不带前缀，针对该差异，当前输出库函数已做宏定义处理，使用原先非 II 形式库函数的函数名称，也可正常使用。故针对本次库函数改动，应用程序源码无需改动；但源文件的数目和名称发生了改变，所以需要对本工程进行修改（即删除原来文件，添加当前新的文件）。具体文件差异参见下表：

文件类型	v1.2.2	v1.2.0
源文件	rn8xxx_II_eeprom.c rn8xxx_II_flash.c rn8xxx_II_rtc_lib.c rn8xxx_II_sysclk.c rn8xxx_II_sysoption.c rn8xxx_II_utils.c rn8xxx_II_sipeeprom.c rn8xxx_II_clktrim.c rn8xxx_II_gpadc_lib.c（新增） rn8xxx_II_emu_lib.c（新增）	eeepromProgram.c nvm.c rtc.c sysctrl.c system_cfg_update.c sysupdate.c utils.c sip_eeprom.c clk_trim.c
头文件	rn8xxx_II_eeprom.h rn8xxx_II_flash.h rn8xxx_II_rtc_lib.h rn8xxx_II_sysclk.h rn8xxx_II_sysoption.h	rn8xxx_v2_lib.h sysctrl_reg.h utils.h

	rn8xxx_ll_utils.h rn8xxx_ll_sipeeprom.h rn8xxx_ll_clktrim.h rn8xxx_ll_gpadc_lib.h rn8xxx_ll_emu_lib.h rn8xxx_ll_devices.h rn8xxx_v2_lib.h	
--	--	--

库函数 V1.2.6 用户更新指引

本用户更新指引仅适用于本文档“修订记录 Rev3.0”所述更改内容。

库函数 V1.2.6 完善了对内置库函数使用，并增加了内置库函数和库函数源码统一接口的支持，具体改动点如下：

- 1、增加源文件 `rn8xxx_ll_lib.c`；
- 2、库函数的使用依赖 CMSIS 文件。

库函数 V1.2.6 使用步骤：

- 1、更新 CMSIS 文件到当前最新版本，即 `pack_v1.1.12` 中的 CMSIS；
- 2、工程中新增库函数源码文件 `rn8xxx_ll_lib.c`；
- 3、在 `renergy_chip_config.h` 文件中选择所需的芯片系列号。
- 4、删除原来工程中芯片系列号的宏定义

注：如需使用内置库函数，请参考下一章节“芯片内置库函数使用方法说明”

芯片内置库函数使用方法说明

为了给客户尽可能多的可用 FLASH 资源，我司在单相 SOC V2/V3 版、三相 SOC V2 版及 MCU V2/V3 版芯片内集成了库函数源代码。如需使用内置库函数，需进行以下操作：

1、更新 CMSIS 文件到当前最新版本，即 pack_v1.1.12 中的 CMSIS。对应版本文件中增加了文件 renergy_chip_config.h。

2、更新库函数到 V1.2.6。该版本库函数中增加了一个源文件 rn8xxx_ll_lib.c，以提供内置库函数校验接口。

3、工程中添加锐能微发布的库函数源码文件。

4、代码中增加检测内置库函数有效性的函数调用 LL_LIB_Check()，若该函数返回无效，请勿调用内置库函数。

5、在 renergy_chip_config.h 文件中选择所需的芯片系列号，并开启宏 NVRLIB_EN。
例如，芯片 RN8318 V2 版、RN8611 V2 版、RN8613 V2 版，启用内置库函数，均可按下面配置：

```
#define RN831x_RN861x_MCU_V2
```

```
#define NVRLIB_EN
```

6、函数调用，调用方法与使用源码一致。如时钟切换库函数为例：

```
LL_SYSCLK_SysModeChg(Clock_PLL_7M4, Clock_Div_1);
```

注 1：内置库函数，无法做升级处理。如果 FLASH 空间大小足够，推荐使用库函数源码，方便后续升级。

注 2：由于芯片内部 NVR 空间有限，内置库函数，不包含时钟调整等库函数接口。下面列出，不支持内置的库函数接口：

序号	函数类别	函数名	函数说明
1	时钟调整函数	LL_CLKTRIM_RchTrimByLosc	RCH 校准函数
2		LL_CLKTRIM_RclTrimByRch	RCL 校准函数
3		LL_CLKTRIM_SysClkMonitorAsync	系统时钟异步监控函数
4		LL_CLKTRIM_SysClkMonitorAsyncStatus	系统时钟异步监控结果获取函数
5		LL_CLKTRIM_SysClkMonitorSync	系统时钟同步监控函数
6		LL_CLKTRIM_LoscStopCheck	LOSC 停振检测函数
7	合封 EEPROM 操作函数	LL_SIP EEPROM_Init	合封 EEPROM 初始化函数
8		LL_SIP EEPROM_Rst	合封 EEPROM 复位函数
9		LL_SIP EEPROM_Read	合封 EEPROM 数据读取函数
10		LL_SIP EEPROM_Write	合封 EEPROM 写函数

1 系统模式切换库函数

1.1 函数接口

函数接口：

eSysclkRet_TypeDef LL_SYCLK_SysModeChg(eSysclkMode_TypeDef mode, eSysclkDiv_TypeDef cdiv)
void LL_SYCLK_FastSysModeChg(eSysclkMode_TypeDef mode)
void LL_SYCLK_NvmActive(void);
void LL_SYCLK_NvmSleep(void);

1.2 函数说明

时钟切换库函数

- ✓ 函数说明：时钟切换
- ✓ 函数原型：eSysclkRet_TypeDef LL_SYCLK_SysModeChg(eSysclkMode_TypeDef mode, eSysclkDiv_TypeDef cdiv)
- ✓ 参数说明：

时钟频率选择参数 mode：

```
typedef enum
{
    Clock_Losc = 0x00,           //外部低频 LOSC
    Clock_RC_1M8 = 0x01,        //内部 RC 1.8432MHz
    Clock_PLL_7M4 = 0x02,        //内部 PLL 7.3728MHz
    Clock_Hosc = 0x03,           //外部高频时钟
    Clock_PLL_14M7 = 0x05,       //内部 PLL14.7456MHz
    Clock_RC_29M5 = 0x06,       //内部 RC29.4912MHz
    Clock_PLL_29M5 = 0x07,       //内部 PLL29.4912MHz
    Clock_RCL_32K = 0x08,        //内部 RCL32.768KHz
    Clock_PLL_58M9_DIV2 = 0x09,
    Clock_CHG_MAX = 0x0a,
} eSysclkMode_TypeDef;

;
分频系数 div:
typedef enum
{
    Clock_Div_1 = 0x00, //不分频
    Clock_Div_2 = 0x01, //2 分频
```

```

Clock_Div_4 = 0x02, //4 分频
Clock_Div_8 = 0x03, //8 分频
} eSysclkDiv_TypeDef;
;
✓ 返回值说明:
    返回时钟切换成功或失败

```

```

typedef enum
{
    SYSCLK_PASS = 0, //时钟切换成功
    SYSCLK_FAIL = 1 //时钟切换失败, 参数配置不合理
} eSysclkRet_TypeDef;

```

系统时钟配置表

模式	值	主频点	不分频	二分频	四分频	八分频
			div=0	div=1	div=2	div=3
Clock_Losc	0	32.768KHz	支持	不支持	不支持	不支持
Clock_RC_1M8	1	1.8432MHz	支持	不支持	不支持	不支持
Clock_PLL_7M4	2	7.3728MHz	支持	支持	支持	不支持
Clock_Hosc	3	7.3728MHz	支持	支持	支持	不支持
		14.7456MHz	支持	支持	支持	支持
		29.4912MHz	支持	支持	支持	支持
Clock_PLL_14M7	5	14.7456MHz	支持	支持	支持	支持
Clock_RC_29M5	6	29.4912MHz	支持	支持	支持	支持
Clock_PLL_29M5	7	29.4912MHz	支持	支持	支持	支持
Clock_RCL_32K	8	32.3KHz	支持	不支持	不支持	不支持
Clock_PLL_58M9_DIV2	9	29.4912MHz	支持	支持	支持	支持

备注:

1、Clock_RC_1M8、Clock_RC_29M5、Clock_RCL_32K 为芯片内部振荡器, 表格中的主频点为典型值, 该值具有一定的离散度, 具体的参数指标参见产品用户手册。

2、当期望切换到高频 (HOSC) 模式, 需要用户根据硬件电路先配置当前晶振的振荡频率, 目前支持 7.3728MHz, 14.7456MHz 和 29.4912MHz 三种晶振。如硬件电路的晶振为 29.4912MHz, 期望配置 7.3728MHz 的系统时钟:

```

SYSCTL->SYS_PS = 0x82;
SYSCTL->OSC_CTRL2 = 0x13;
sta = SysModeChg(3, 2);

```

3、由于低功耗 LOSC 电路第一次上电起振需要 0.5s 时间, 因此当期望配置为 LOSC 或 PLL 模式时, 建议用户判断 SYSCTL->SYS_RST 寄存器的上下电复位标志位, 如果置起, 表示是第一次上电, 软件需要增加 0.5s 延时, 等待 LOSC 稳定后再调用系统模式切换库函数。

4、RN8213 (C 版), RN8211B (C 版), RN861x 和 RN8318 (V1 和 V2 版本) 支持 14.7456MHz PLL 模式, 其他型号不支持这两种模式;

5、RN861x (V2 版本)、RN831x (V2 版本)、RN821x (V2 版本) 和 RN721x (V2 版本) 支持 PLL 和 RCH 的 29.4912Mhz 模式 (Clock_RC_29M5、Clock_PLL_29M5 和

Clock_PLL_58M9_DIV2)；其中，Clock_RC_29M5 的 29.4912MHz 有一定的离散度，推荐仅作系统高频时钟的备份时钟使用；选择 Clock_RC_29M5 时钟模式后，要求用户必须定时调用 RCH 时钟调整函数 RchTrimByLosc 去调整芯片内部 RCH，定时周期建议选择 1 秒到 1 分钟之间。

6、针对三相 soc 系列，即 RN2026/RN2028/R7326，晶振的最高典型频率为 32.768Mhz，并在该频点下分频。其最高典型频率，与 MCU/单相 SOC 系列的 29.4912Mhz 有差异，客户在切换时钟时，也按“eSysCtrlClockMode_TypeDef”结构体即可，实际是按 32.768Mhz 切换或者往下分频的。

7、通过 LL_RTC_ClkSel 函数将 RTC 等模块时钟选择为 RCL 后，不能直接将系统主频切换到 PLL。必须先通过 LL_RTC_ClkSel 函数，将 RTC 时钟切换为 LOSC，才能进一步切换。具体参见文档<锐能微 MCU V2 版/V3 版芯片外部晶振异常应用处理说明>。

快速时钟切换函数

- ✓ 函数说明：快速时钟切换
- ✓ 函数原型：void LL_SYSClk_FastSysModeChg(eSysclkMode_TypeDef mode)
- ✓ 参数说明：

时钟频率选择参数 mode：

```
typedef enum
{
    Clock_Losc = 0x00,           //外部低频 LOSC
    Clock_RC_1M8 = 0x01,        //内部 RC 1.8432MHz
    Clock_PLL_7M4 = 0x02,        //内部 PLL 7.3728MHz
    Clock_Hosc = 0x03,           //外部高频时钟
    Clock_PLL_14M7 = 0x05,       //内部 PLL14.7456MHz
    Clock_RC_29M5 = 0x06,        //内部 RC29.4912MHz
    Clock_PLL_29M5 = 0x07,       //内部 PLL29.4912MHz
    Clock_RCL_32K = 0x08,        //内部 RCL32.768KHz
    Clock_PLL_58M9_DIV2 = 0x09,
    Clock_CHG_MAX = 0x0a,
} eSysCtrlClockMode_TypeDef;
```

- ✓ 返回值说明：无

注意：快速时钟切换函数。只能用于 LOSC32K 和 RC1M8 之间切换。该函数用于快速进入和退出低功耗模式，该函数不做 HOSC/PLL 时钟关闭的操作，因此需要使用 SysModeChg 进入 LOSC32K/RC1M8 后才能调用此函数以确保 HOSC/PLL 被正确关闭。

全失压激活函数

- ✓ 函数说明：激活全失压测量功能，系统主频切换到 LOSC 后，启动全失压测量前，需调用该函数，将 LDO 切换到大功耗模式，以便全失压模块可正常工作
- ✓ 函数原型：void LL_SYSCLK_NvmActive(void);
- ✓ 参数说明：无
- ✓ 返回值说明：无

全失压休眠函数

- ✓ 函数说明：全失压测量完成后，MCU 休眠前需调用该函数，将 LDO 切换到低功耗模式，以降低休眠功耗
- ✓ 函数原型：void LL_SYSCLK_NvmSleep(void);
- ✓ 参数说明：无
- ✓ 返回值说明：无

2 库函数版本获取函数

版本获取函数仅适用于二进制版本库函数，源码访问该接口无意义。

2.1 函数接口

函数接口：

unsigned int LL_SYSCLK_GetLibVersion(void);
unsigned int LL_SYSCLK_GetCodeVersion(void);

2.2 函数说明

库函数版本获取函数

- ✓ 函数说明：获取库函数版本
- ✓ 函数原型：unsigned int LL_SYSCLK_GetLibVersion(void)
- ✓ 参数说明：

无

✓ 返回值说明:

unsigned int: 库函数版本号, 32 位无符号数, 每 8bit 为一个域, 按照从高到低的顺序排布, 0x[00][xx][yy][zz], 其中 x, y, z 分别表示软件版本号的大版本号, 中版本号和小版本号, 如 v1.2.10 对应 0x0001020A。如库函数返回 0x00010001, 表示当前库函数版本号 V1.0.1。

库函数源码版本获取函数

✓ 函数说明: 获取库函数源码版本

✓ 函数原型: `unsigned int LL_SYCLK_GetCodeVersion(void)`

✓ 参数说明:

无

✓ 返回值说明:

unsigned int: 库函数源码版本号, 32 位无符号数, 对应内部代码版本管理号。如 0x00003c18 表示版本号为 15384。该版本号用于库函数源码追溯用, 客户可不关注。

3 FLASH 操作库函数

3.1 函数接口

函数接口:

<code>eFlashRet_TypeDef LL_FLASH_PageErase(unsigned int pg);</code>
<code>eFlashRet_TypeDef LL_FLASH_SectorErase(unsigned int sec);</code>
<code>eFlashRet_TypeDef LL_FLASH_Program(unsigned int dst_addr, unsigned int src_addr, unsigned int len);</code>
<code>void LL_FLASH_CacheOn(void);</code>
<code>void LL_FLASH_CacheOff(void);</code>
<code>eEepromRet_TypeDef LL_EEPROM_Program(unsigned int dst_addr, unsigned int src_addr, unsigned int len);</code>
<code>void LL_EEPROM_Standby(void);</code>
<code>void LL_EEPROM_Wakeup(void);</code>

3.2 函数说明

MCU/SOC 系列产品的内置 FLASH 采用页结构, 其中对于小于 256KB 容量的芯片, 页 (page) 大小为 128 字节, 32 页为一块 (sector); 大于等于 256KB 容量的芯片, 页 (page)

大小为 256 字节，64 页为一块（sector）；V2/V3 版本 MCU 产品和 V2 SOC 产品，页大小为 512 字节；V2 版本 MCU 和 V2 版本 SOC 32 页为一块（sector），V3 版本 MCU 和 V3 版本 SOC 16 页为一块（sector）；锐能微各芯片内嵌的 FLASH 大小及页大小等相关参数参见“表 3.1 锐能微芯片 FLASH 的特性参数”：

芯片型号	FLASH SIZE	FLASH MEMORY	PAGE SIZE	PAGE ERASE TIME	PAGE PROGRAM TIME
RN8211	128KB	0x00000000~0x0001FFFF	128Byte	8ms	5ms
RN8211B	128KB				
RN8213	128KB				
RN8215	192KB	0x00000000~0x0002FFFF			
RN8213B	128KB	0x00000000~0x0001FFFF			
RN8312	192KB	0x00000000~0x0002FFFF			
RN8316	192KB				
RN8211B (C)	128KB	0x00000000~0x0001FFFF			
RN8213 (C)	128KB				
RN8612	128KB				
RN8611	256KB	0x00000000~0x0003FFFF	256Byte	5ms	3ms
RN8613	256KB				
RN8615	512KB	0x00000000~0x0007FFFF			
RN8318	512KB				
RN8217	512KB				
RN8213B (D)	512KB				
RN2023	128KB	0x00000000~0x0001FFFF	128Byte	5ms	3ms
RN2025	256KB	0x00000000~0x0003FFFF	256Byte	5ms	3ms
RN2026	256KB				
RN7326	256KB				
RN8611 (V2)	512KB	0x00000000~0x0007FFFF	512Byte	3ms	3ms
RN8613 (V2)	512KB				
RN8615 (V2)	512KB				
RN8318 (V2)	512KB				
RN2028 (V2)	512KB	0x00000000~0x0007FFFF	512Byte	3ms	3ms
RN2026 (V2)	512KB				
RN8213B (V2)	512KB				
RN8217 (V2)	512KB				
RN8215 (V2)	512KB				
RN7213 (V2)	512KB				
RN8213 (V2)	256KB	0x00000000~0x0003FFFF			
RN8211B (V2)	256KB				
RN8613 (V3)	256KB	0x00000000~0x0003FFFF	512Byte	3ms	3ms
RN8610 (V3)	256KB				
RN8213 (V3)	256KB				
RN8211B (V3)	256KB				

RN8211 (V3)	256KB				
-------------	-------	--	--	--	--

表 3.1 锐能微芯片 FLASH 的特性参数

非易失性存储器操作接口支持对 FLASH 的片擦除，页擦除，以及编程等操作。对 FLASH 进行编程前务必要先擦除。由于对 FLASH 的擦写是以页为单位，因此建议对 FLASH 的编程传递的参数使用页对齐，如果没有页对齐，页内未操作到的内容将保持擦除后的 0 数据。

FLASH 页擦除函数

- ✓ 函数说明：完成对 FLASH 的某一页的擦除操作
- ✓ 函数原型：eFlashRet_TypeDef LL_FLASH_PageErase(unsigned int pg);
- ✓ 参数说明：

unsigned int pg: 擦除的页地址，支持的范围为：0 ~ flash_max_size*1024/flash_page_size - 1

- ✓ 返回值说明：
返回页擦除操作结果

```
typedef enum
{
    NVM_PASS = 0,      //操作成功
    INVALID_PAGE = 8,  //页地址无效
} eNvmRet_TypeDef;
```

FLASH 块擦除函数

- ✓ 函数说明：完成对 FLASH 的某一块的擦除操作
- ✓ 函数原型：eFlashRet_TypeDef LL_FLASH_SectorErase(unsigned int sec);
- ✓ 参数说明：

unsigned in: 擦除的页地址，支持的范围为：0 ~ flash_sector_num-1

- ✓ 返回值说明：
返回 FLASH 块擦除结果

```
typedef enum
{
    NVM_PASS = 0, //操作成功
} eNvmRet_TypeDef;
```

FLASH 编程函数

- ✓ 函数说明：完成对 FLASH 的编程操作
- ✓ 函数原型：eFlashRet_TypeDef LL_FLASH_Program(unsigned int dst_addr, unsigned int src_addr, unsigned int len);
- ✓ 参数说明：

unsigned int dst_addr: Flash 的编程地址，要求地址按**页对齐**

unsigned int src_addr: Flash 编程的源数据地址，支持任意地址；推荐使用**字对齐**。

unsigned int len: Flash 编程的数据长度，依据源地址字对齐和非字对齐有不同要求：
源地址字对齐时，长度要求为页大小的整数倍或小于页字节数（待编程的最后一页）；
源地址非字对齐时，无编程长度限制，函数内做数据搬移，编程速度会下降；推荐使用源地址字对齐方式。

- ✓ 返回值说明：

返回 FLASH 的编程操作结果

typedef enum

```
{  
    NVM_PASS = 0,                //操作成功  
    COUNT_ERROR = 6,            //编程长度超限  
    INVALID_PAGE = 8,           //页地址无效  
    SRC_ADDR_NOT_ALIGN = 10,     //源地址不对齐  
    SRC_ADDR_NOT_MAPPED = 11,   //源地址超范围  
    DST_ADDR_NOT_ALIGN = 12,    //目标地址不对齐，即目标地址不是页起始地址  
    DST_ADDR_NOT_MAPPED = 13    //目标地址超范围，目标地址小于 FLASH 起始地址  
    或者目标地址+len 超过 FLASH 起始地址+FLASH 大小  
} eNvmRet_TypeDef;
```

- ✓ 应用说明：

1) 芯片内 FLASH 的最小操作单位是一页，编程前需要做擦除操作，否则无法将数据有效的编程到 FLASH 内；如果要将 FLASH 作为数据区使用，需先读回对应页数据缓存，修改所需修改地址内容后，依次调用 FLASH 页擦除函数和 FLASH 编程函数接口。

2) V2/V3 版 SOC 和 MCU 产品，Flash 编程函数比原先产品所用堆栈增加 1.1KB，用户程序需关注其堆栈分配，确保有足够大的空间，防止堆栈溢出。

cache 打开函数

- ✓ 函数说明：开启 cache 功能
- ✓ 函数原型：void LL_FLASH_CacheOn(void)
- ✓ 参数说明：

无

- ✓ 返回值说明：

无

提示：

1、芯片进入低功耗必须打开 cache 功能，否则无法进入低功耗状态。RN8613, RN8615, RN8318 V1 版本应用程序可以调用库函数，也可以直接操作寄存器来使能 cache 功能，寄存器地址是 0x50014060，写入的数据是 0xfeed0001；其他系列芯片请调用该函数。

2、V2 版本 MCU 及三相 SOC 产品，没有 cache，不需调用该函数。调用后不起作用。

cache 关闭函数

- ✓ 函数说明：关闭 cache 功能
- ✓ 函数原型：void LL_FLASH_CacheOff(void)
- ✓ 参数说明：
 无
- ✓ 返回值说明：
 无

提示：

1、RN8613, RN8615, RN8318 V1 版本应用程序可以调用库函数，也可以直接操作寄存器来关闭 cache 功能，寄存器地址是 0x50014060，写入的数据是 0xfeed0000；其他系列芯片请调用该函数。

2、V2 版本 MCU 及三相 SOC 产品，没有 cache，不需调用该函数。调用后不起作用。

4 EEPROM 操作库函数

4.1 函数接口

函数接口：

eEepromRet_TypeDef LL_EEPROM_Program(unsigned int dst_addr, unsigned int src_addr, unsigned int len);
void LL_EEPROM_Standby(void);
void LL_EEPROM_Wakeup(void);

4.2 函数说明

锐能微各芯片内嵌的 EEPROM 大小及页大小等相关参数不同，具体见“表 3.2 锐能微芯片内嵌 EEPROM 的特性参数”。

芯片型号	EEPROM SIZE	PAGE SIZE	PAGE ERASE TIME	PAGE PROGRAM TIME	EEPROM MEMORY
RN8211	32KB	64Byte	2.5ms	1.5ms	0x08000000 ~ 0x08007fff
RN8211B					
RN8213					

RN8215					
RN8213B					
RN8312					
RN8316					
RN8211B (C)	32KB	128Byte	2. 5ms	1. 5ms	
RN8213 (C)					
RN8612					

表 3.2 锐能微芯片内嵌 EEPROM 的特性参数

RN8611、RN8613、RN8615、RN8318 无内置 EEPROM。

RN8213（C 版）、RN8211B（C 版）、RN8612 内置 EEPROM 的页（page）大小为 128 字节，块（sector）大小为 64 页，这些型号的 EEPROM 本身并不支持字节编程，只支持字编程，**虽然库函数支持了字节编程，但是对频繁擦写的数**据一定要采用字对齐，**否则就不能满足 100 万次的编程次数要求。**其他型号内置 EEPROM 的页大小为 64 字节，块大小为 32 页，字节编程支持 100 万次。

非易失性存储器操作接口支持对 EEPROM 的片擦除，页擦除以及编程等操作。EEPROM 的编程函数包括了擦除动作，支持任意字节长度的编程操作。对于无内置 EEPROM 的芯片，调用相应的函数不发生任何行为。

EEPROM 编程函数

- ✓ 函数说明：完成对 EEPROM 的编程操作
- ✓ 函数原型：eEepromRet_TypeDef LL_EEPROM_Program(unsigned int dst_addr, unsigned int src_addr, unsigned int len);
- ✓ 参数说明：
 - unsigned int dst_addr: Eeprom 的编程地址
 - unsigned int src_addr: Eeprom 编程的原数据地址
 - unsigned int len: Eeprom 编程的数据长度
- ✓ 返回值说明：

返回 EEPROM 的编程操作结果

```
typedef enum
{
    NVM_PASS = 0,                //操作成功
    COUNT_ERROR = 6,             //编程长度超限
    INVALID_PAGE = 8,            //页地址无效
    SRC_ADDR_NOT_ALIGN = 10,     //源地址不对齐
    SRC_ADDR_NOT_MAPPED = 11,    //源地址超范围
    DST_ADDR_NOT_ALIGN = 12,     //目标地址不对齐，即目标地址不是页起始地址
    DST_ADDR_NOT_MAPPED = 13    //目标地址超范围，目标地址小于 EEPROM 起始地址或者目标地址+len 超过 EEPROM 起始地址+EEPROM 大小
} eNvmRet_TypeDef;
```

EEPROM 进入低功耗函数

- ✓ 函数说明：EEPROM 进入低功耗状态
- ✓ 函数原型：void LL_EEPROM_Standby(void);
- ✓ 参数说明：
无
- ✓ 返回值说明：
无

EEPROM 唤醒函数

- ✓ 函数说明：EEPROM 唤醒退出低功耗状态
- ✓ 函数原型：void LL_EEPROM_Wakeup(void);
- ✓ 参数说明：
无
- ✓ 返回值说明：
无

5 RTC 操作库函数

5.1 函数接口

函数接口：

unsigned short LL_RTC_ReadDota(void);
eRtcRet_TypeDef LL_RTC_WriteDota(unsigned short dota);
void LL_RTC_ClkSel(rtc_clk_e clk_sel);

5.2 函数说明

RTC DOTA 参数读函数

- ✓ 函数说明：读取 RTC 模块的 DOTA 参数
- ✓ 函数原型：unsigned short LL_RTC_ReadDota(void);
- ✓ 参数说明：
无
- ✓ 返回值说明：
unsigned short dota: RTC 模块的 DOTA 参数值

RTC DOTA 参数写函数

- ✓ 函数说明：配置 RTC 模块的 DOTA 参数
- ✓ 函数原型：eRtcRet_TypeDef LL_RTC_WriteDota(unsigned short dota);
- ✓ 参数说明：

unsigned short dota: RTC 模块的 DOTA 参数写入值

- ✓ 返回值说明：

RTC 模块 DOTA 参数写入成功或失败

typedef enum

{

RTC_PASS = 0, //返回成功

RTC_FAIL = 1, //返回错误, Vbat 电源域工作异常写入失败

} eRtcRet_TypeDef;

说明：

1、dota 表示初始频率偏差校正值，格式为二进制补码，bit15:13 固定为 0，bit12 为符号位；

计算方法如下：

若 $\sigma_0 \geq 0$ ，则 $dota = \text{round}(\sigma_0 * 32)$ ；

若 $\sigma_0 < 0$ ，则 $dota = 2^{13} + \text{round}(\sigma_0 * 32)$ ；

其中 σ_0 单位为 ppm， σ_0 可表示范围：-128ppm~127.96875ppm，round() 为四舍五入公式。

dota 分辨度为 0.03125ppm。大部分 32768 晶振，常温典型误差为 ± 20 ppm，最大值为 ± 30 ppm。举例，

初始频率偏差 σ_0 为 0.1823(s/day)，将单位转为 ppm，则 σ_0
= $0.1823 / (60 * 60 * 24) * 10^6 = 2.11$ ppm；则校正值 $dota = \text{round}(2.11 * 32) = 0x0044$ ；

若初始频率偏差为 σ_0 为 -0.1823(s/day)，即 -2.11ppm，则校正值
 $dota = 2^{13} + \text{round}(-2.11 * 32) = 8124 = 0x1FBC$ ；

RTC 时钟源选择函数

- ✓ 函数说明：RTC 时钟源选择函数
- ✓ 函数原型：void LL_RTC_ClkSel(rtc_clk_e clk_sel);
- ✓ 参数说明：无

typedef enum {

RTC_CLK_IS_LOSC = 0, //选择 RTC 的时钟源为 LOSC

RTC_CLK_IS_RCL = 1 //选择 RTC 的时钟源为 RCL

} rtc_clk_e;

- ✓ 返回值说明：

无

- ✓ 应用说明：调用该函数将 RTC 时钟选择为 RCL 后，不能将系统主频切换到 PLL。

具体参见文档<锐能微 MCU V2 版/V3 版芯片外部晶振异常应用处理说明>。

6 芯片参数校验库函数

芯片参数管理库函数包括系统参数、RTC 配置参数、WDT 配置参数、RCH 校正参数及 RCL 校正参数的校验库函数，用于进行相应参数管理；以上芯片参数管理库函数，均以芯片内部固化参数为标准，若客户没有特殊需求，建议调用所有的芯片参数管理函数，相关函数参见 6.1 函数接口。若客户有特殊需求，如需使用特定的 RTC 配置参数，则不能再调用 RTC 配置参数校验库函数 `LL_SYSOPTION_RtcConfCheck`；或者系统内存在调用时钟调整函数实时修改 RCH 的，请勿调用 RCH 校正参数校验库函数 `LL_SYSOPTION_RchConfCheck`。

6.1 函数接口

函数接口：

<code>eSysoptionRet_TypeDef LL_SYSOPTION_SysConfCheck(void);</code>
<code>eSysoptionRet_TypeDef LL_SYSOPTION_RtcConfCheck(void);</code>
<code>eSysoptionRet_TypeDef LL_SYSOPTION_WdtConfCheck(void);</code>
<code>eSysoptionRet_TypeDef LL_SYSOPTION_RchConfCheck(void);</code>
<code>eSysoptionRet_TypeDef LL_SYSOPTION_RclConfCheck(void);</code>
<code>eSysoptionRet_TypeDef LL_SYSOPTION_FlashVoltageTrim(void);</code>

6.2 函数说明

系统配置参数校验库函数

- ✓ 函数说明：校验并配置芯片系统配置参数
- ✓ 函数原型：`eSysoptionRet_TypeDef LL_SYSOPTION_SysConfCheck(void);`
- ✓ 参数说明：
无
- ✓ 返回值说明：
芯片系统配置参数校验及配置结果。
`typedef enum`
{
 `SYSOPTION_PASS = 0`, //参数校验通过

```
SYSOPTION_FAIL = 1    //参数校验失败
} eSysoptionRet_TypeDef;
```

- ✓ 应用说明：芯片复位后，Bootloader 内部启动代码会自动配置芯片参数，但可能存在配置不成功的风险。比如，写入 VBAT 电源域的配置寄存器（RTC 温补曲线等参数）依赖于 VBAT 电源稳定，但是当 VBAT 上电时间比 VCC 慢，这时 VBAT 电源还不稳定，可能就会配置失败。

大部分的芯片配置寄存器只在复位后配置一次，假如系统因某种原因导致寄存器值被篡改，存在较大的运行风险。

因此，用户必须调用芯片参数管理函数，调用方式如下：

1、复位后，必须先调用该函数，才能进行其他操作；

2、程序正常运行时要求用户必须定时调用该函数去校验芯片内部的系统参数，定时周期可以选择 1 秒到 1 分钟之间。休眠后，建议定时调用该函数，为平衡功耗，定时周期可选择 10 分钟以上。

3、当客户有 BOOT 程序时，BOOT 程序也要执行上述 1、2 两步操作。

该函数会对芯片内部系统参数进行检查，发现异常后会执行恢复操作，恢复后再次检查；若还是错误，则返回 SYSCTRL_FAIL，表示芯片异常。

RTC 配置参数校验库函数

- ✓ 函数说明：校验芯片内部 RTC 配置参数
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_RtcConfCheck(void);
- ✓ 参数说明：

无

- ✓ 返回值说明：
芯片 RTC 配置参数校验结果。

```
typedef enum
{
    SYSOPTION_PASS = 0, //参数校验通过
    SYSOPTION_FAIL = 1  //参数校验失败
} eSysoptionRet_TypeDef;
```

- ✓ 应用说明：参见系统参数校验库函数中的应用说明。
- ✓ 应用说明：该函数会将 RTC 配置参数修改成芯片默认参数值。客户有特定 RTC 配置参数应用的，需自行管理，请勿调用该函数进行 RTC 配置参数管理。

WDT 配置参数校验库函数

- ✓ 函数说明：校验芯片内部 WDT 配置参数
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_WdtConfCheck(void);
- ✓ 参数说明：

无

- ✓ 返回值说明：
芯片 WDT 配置参数校验结果。
typedef enum
{
 SYSOPTION_PASS = 0, //参数校验通过
 SYSOPTION_FAIL = 1 //参数校验失败
} eSysoptionRet_TypeDef;
- ✓ 应用说明：参见系统参数校验库函数中的应用说明。
- ✓ 应用说明：该函数会将 WDT 配置参数修改成芯片默认参数值。客户有特定 WDT 配置参数应用的，需自行管理，请勿调用该函数进行 WDT 配置参数管理。

RCH 校正参数校验库函数

- ✓ 函数说明：校验芯片内部 RCH 校正参数
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_RchConfCheck(void);
- ✓ 参数说明：
 无
- ✓ 返回值说明：
芯片 RCH 校正参数校验结果。
typedef enum
{
 SYSOPTION_PASS = 0, //参数校验通过
 SYSOPTION_FAIL = 1 //参数校验失败
} eSysoptionRet_TypeDef;
- ✓ 应用说明：参见系统参数校验库函数中的应用说明。
- ✓ 应用说明：该函数会将 RCH 校正参数修改成芯片默认参数值。系统内存在调用时钟调整函数实时修改 RCH 的应用场景下，请勿调用该库函数

RCL 校正参数校验库函数

- ✓ 函数说明：校验芯片内部 RCL 校正参数
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_RclConfCheck(void);
- ✓ 参数说明：
 无
- ✓ 返回值说明：
芯片 RCL 校正参数校验结果。
typedef enum
{
 SYSOPTION_PASS = 0, //参数校验通过
 SYSOPTION_FAIL = 1 //参数校验失败
} eSysoptionRet_TypeDef;

- ✓ 应用说明：参见系统参数校验库函数中的应用说明。
- ✓ 应用说明：该函数会将 RCL 校正参数修改成芯片默认参数值。系统内存在调用时钟调整函数实时修改 RCL 的应用场景下，请勿调用该库函数。

FlashVoltageTrim 系统参数配置库函数

- ✓ 函数说明：配置系统关键参数
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_FlashVoltageTrim(void)
- ✓ 参数说明：
无
- ✓ 返回值说明：
系统参数配置结果：

```
typedef enum
{
    SYSOPTION_PASS = 0, //系统参数配置成功
    SYSOPTION_FAIL = 1  //系统参数配置失败
} eSysoptionRet_TypeDef;
```

V2 芯片在进行 FLASH 擦写前至少需要调用一次该函数。该函数为 SysConfCheck 的简化版函数。相对 SysConfCheck 函数，FlashVoltageTrim 函数执行时间短，且不会做芯片内部配置参数做检测，也不会检测到内部配置参数异常时进行恢复操作，即 FlashVoltageTrim 函数只调整芯片的 Flash 工作参数。在用户有修改芯片内部配置参数及关注代码运行时间的地方，建议调用 FlashVoltageTrim。

7 选项字节操作函数

7.1 函数接口

函数接口：

eSysoptionRet_TypeDef LL_SYSOPTION_Dota0CfgSet(unsigned short dota0);
eSysoptionRet_TypeDef LL_SYSOPTION_AlphaCfgSet(unsigned short alpha);
eSysoptionRet_TypeDef LL_SYSOPTION_AlphaCfgSet(unsigned short alphah);
eSysoptionRet_TypeDef LL_SYSOPTION_Xt0CfgSet(unsigned short xt0);
eSysoptionRet_TypeDef LL_SYSOPTION_Remap4Sel(unsigned char sel);
eSysoptionRet_TypeDef LL_SYSOPTION_CpLevelSet(unsigned int cp_level, unsigned int cp_passwd)

eSysoptionRet_TypeDef LL_SYSOPTION_Dota0CfgGet(unsigned short *dota0);
eSysoptionRet_TypeDef LL_SYSOPTION_AlphalCfgGet(unsigned short *alphal);
eSysoptionRet_TypeDef LL_SYSOPTION_AlphahCfgGet(unsigned short *alphah);
eSysoptionRet_TypeDef LL_SYSOPTION_Xt0CfgGet(unsigned short *xt0);
eSysoptionRet_TypeDef LL_SYSOPTION_RemapGet(unsigned short *remap);
eSysoptionRet_TypeDef LL_SYSOPTION_CpLevelGet(unsigned int *cp_level)
eMcuVer_TypeDef LL_SYSOPTION_GetMcuVersion(void);

7.2 函数说明

芯片内置了一个选项字节的区域，当芯片发生复位时，会自动根据选项字节中的内容，配置芯片的相应寄存器，执行设定的功能。选项字节包括对芯片保护，WDT，EMAP 和 RTC 的配置。选项字节编程可以通过锐能微的编程工具进行设置，也可使用本节提供的库函数接口进行设置。以下分别列出相关库函数，及其修改到的选项字节。

Dota0 配置函数

- ✓ 函数说明：将初始频率偏差校正值 Dota0 写入到选项字节对应地址
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_Dota0CfgSet(unsigned short dota0)
- ✓ 参数说明：
unsigned short dota0: 待写入初始频率偏差校正寄存器值

- ✓ 返回值说明：
返回写初始频率偏差校正值到选项字节对应地址的操作结果

```
typedef enum
{
    SYSOPTION_PASS = 0, //写入成功
    SYSOPTION_FAIL = 1 //写入失败
} eSysoptionRet_TypeDef;
```

Alphal 配置函数

- ✓ 函数说明：将晶振低温部分温度系数 Alphal 写入到选项字节对应地址
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_AlphalCfgSet(unsigned short alphal)
- ✓ 参数说明：
unsigned short alphal: 待写入晶振低温部分温度系数寄存器值

- ✓ 返回值说明：
返回写晶振低温部分温度系数 Alphal 寄存器值到选项字节对应地址的操作结果

```
typedef enum
```

```
{
    SYSOPTION_PASS = 0, //写入成功
    SYSOPTION_FAIL = 1 //写入失败
} eSysoptionRet_TypeDef;
```

Alphah 配置函数

- ✓ 函数说明：将晶振高温部分温度系数 Alphah 写入到选项字节对应地址
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_AlphahCfgSet(unsigned short alphah)
- ✓ 参数说明：
unsigned short alphah: 待写入晶振高温部分温度系数寄存器值

- ✓ 返回值说明：
返回写晶振高温部分温度系数寄存器值到选项字节对应地址的操作结果

```
typedef enum
{
    SYSOPTION_PASS = 0, //写入成功
    SYSOPTION_FAIL = 1 //写入失败
} eSysoptionRet_TypeDef;
```

Xt0 配置函数

- ✓ 函数说明：将晶振温度曲线顶点温度 Xt0 写入到选项字节对应地址
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_Xt0CfgSet(unsigned short xt0)
- ✓ 参数说明：
unsigned short xt0: 待写入晶振温度曲线顶点温度 Xt0 寄存器值

- ✓ 返回值说明：
返回写晶振温度曲线顶点温度 Xt0 寄存器值到选项字节对应地址的操作结果

```
typedef enum
{
    SYSOPTION_PASS = 0, //写入成功
    SYSOPTION_FAIL = 1 //写入失败
} eSysoptionRet_TypeDef;
```

Remap 配置函数

- ✓ 函数说明：芯片复位引导区配置函数，将地址映射模式配置到选项字节对应地址
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_Remap4Sel(unsigned char sel)

✓ 参数说明:

unsigned char sel: 地址映射模式字, 0 表示普通模式, 1 表示 remap4 模式

sel=0 时, 将设置 REMAP4 标志, 下次上电引导, 系统将从高端 FLASH 启动;

sel=1 时, 将设置 REMAP0 标志, 下次上电引导, 系统将从低端 FLASH 启动。

✓ 返回值说明:

返回写地址映射模式到选项字节对应地址的操作结果

typedef enum

```
{  
    SYSOPTION_PASS = 0, //引导区配置成功  
    SYSOPTION_FAIL = 1 //引导区配置失败  
} eSysoptionRet_TypeDef;
```

提示:

支持主备倒换功能的芯片型号包括: RN8215, RN8312, RN8213B, RN8611, RN8613, RN8615, RN8318。

如果使用主备倒换功能, FLASH 将被拆分成高低两个独立的 FLASH, 容量各为总容量的一半; 如果启用了主备倒换机制, 即选择 REMAP4, 系统在上电引导时, 将会从高端的 FLASH 引导启动; 否则, 将从低端的 FLASH 启动。

保护等级配置函数

✓ 函数说明: 设置芯片的安全保护等级

函数原型: eSysoptionRet_TypeDef LL_SYSOPTION_CpLevelSet(unsigned int cp_level, unsigned int cp_passwd)

✓ 参数说明:

unsigned int cp_level: 芯片的安全保护等级

unsigned int cp_passwd: ISP 访问密码

✓ 返回值说明:

返回保护等级设置的结果

typedef enum

```
{  
    SYSOPTION_PASS = 0, //配置成功  
    SYSOPTION_FAIL = 1 //配置失败  
} eSysoptionRet_TypeDef;
```

Dota0 获取函数

✓ 函数说明: 获取选项字节中初始频率偏差校正值 Dota0

✓ 函数原型: eSysoptionRet_TypeDef LL_SYSOPTION_Dota0CfgGet(unsigned short *dota0)

✓ 参数说明:

unsigned short *dota0: 获取到的初始频率偏差校正值指针

✓ 返回值说明:

返回获取初始频率偏差校正值的操作结果

typedef enum

```
{  
    SYSOPTION_PASS = 0, //获取成功  
    SYSOPTION_FAIL = 1 //获取失败  
} eSysoptionRet_TypeDef;
```

Alphal 获取函数

✓ 函数说明: 获取选项字节中晶振低温部分温度系数 Alphal

✓ 函数原型: eSysoptionRet_TypeDef LL_SYSOPTION_AlphalCfgGet(unsigned short *alphal)

✓ 参数说明:

unsigned short *alphal: 获取到的晶振低温部分温度系数 Alpha 指针

✓ 返回值说明:

返回获取晶振低温部分温度系数 Alphal 的操作结果

typedef enum

```
{  
    SYSOPTION_PASS = 0, //获取成功  
    SYSOPTION_FAIL = 1 //获取失败  
} eSysoptionRet_TypeDef;
```

Alphah 获取函数

✓ 函数说明: 获取选项字节中晶振高温部分温度系数 Alphah

✓ 函数原型: eSysoptionRet_TypeDef LL_SYSOPTION_AlphahCfgGet(unsigned short *alphah)

✓ 参数说明:

unsigned short *alphah: 获取到的晶振高温部分温度系数 Alphah 指针

✓ 返回值说明:

返回获取晶振高温部分温度系数 Alphah 的操作结果

typedef enum

```
{  
    SYSOPTION_PASS = 0, //获取成功  
    SYSOPTION_FAIL = 1 //获取失败  
} eSysoptionRet_TypeDef;
```

Xt0 获取函数

- ✓ 函数说明：获取选项字节中晶振温度曲线顶点温度 Xt0
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_Xt0CfgGet(unsigned short *xt0)
- ✓ 参数说明：
unsigned short *xt0：获取到的晶振温度曲线顶点温度 Xt0 指针

- ✓ 返回值说明：
返回获取晶振温度曲线顶点温度 Xt0 的操作结果

```
typedef enum
{
    SYSOPTION_PASS = 0, //获取成功
    SYSOPTION_FAIL = 1 //获取失败
} eSysoptionRet_TypeDef;
```

Remap 获取函数

- ✓ 函数说明：获取选项字节中地址映射模式
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_RemapGet(unsigned short *remap)
- ✓ 参数说明：
unsigned short *remap：获取到的地址映射模式指针

- ✓ 返回值说明：
返回获取地址映射模式的操作结果

```
typedef enum
{
    SYSOPTION_PASS = 0, //获取成功
    SYSOPTION_FAIL = 1 //获取失败
} eSysoptionRet_TypeDef;
```

保护等级获取函数

- ✓ 函数说明：获取芯片当前的保护等级
- ✓ 函数原型：eSysoptionRet_TypeDef LL_SYSOPTION_CpLevelGet(unsigned int *cp_level)
- ✓ 参数说明：
unsigned int *cp_level：获取到的芯片保护等级指针

- ✓ 返回值说明：
返回获取芯片保护等级的操作结果

```
typedef enum
{
    SYSOPTION_PASS = 0, //获取成功
    SYSOPTION_FAIL = 1 //获取失败
} eSysoptionRet_TypeDef;
```

MCU 版本获取函数

- ✓ 函数说明：获取锐能微 MCU 芯片的版本信息，该函数不支持 SOC 产品的版本信息返回
- ✓ 函数原型：eMcuVer_TypeDef LL_SYSOPTION_GetMcuVersion(void)
- ✓ 参数说明：无
- ✓ 返回值说明：

MCU 的版本信息

```
typedef enum {  
    MCU_VER_V21 = 0x21,    //V21 版本  
    MCU_VER_V22 = 0x22,    //V22 版本  
    MCU_VER_V3  = 0x30,    //V3 版本  
    MCU_VER_INVALID = 0xff  //不能识别的版本  
} eMcuVer_TypeDef;
```

8 合封 EEPROM 操作函数

8.1 函数接口

函数接口：

SIPEEPROM_RET_TYPEDEF LL_SIPEEPROM_Read(unsigned int addr, unsigned char *data, unsigned len);
SIPEEPROM_RET_TYPEDEF LL_SIPEEPROM_Write(unsigned int addr, const unsigned char *data, unsigned int len);
void LL_SIPEEPROM_Init(void);
void LL_SIPEEPROM_Rst(void);

8.2 函数说明

对于内部合封了 EEPROM 的芯片，可用本章提供的函数接口读写合封 EEPROM，本章的函数接口不支持内置。下面具体说明合封 EEPROM 操作函数。

合封 EEPROM 数据读取函数

- ✓ 函数说明：读取合封 EEPROM 的数据

- ✓ 函数原型：SIPEEPROM_RET_TYPEDEF LL_SIPEEPROM_Read(unsigned int addr, unsigned char *data, unsigned len)

- ✓ 参数说明：

unsigned int addr: 待读取合封 EEPROM 的地址

unsigned char *data: 读取到的数据存放指针

unsigned len: 待读取数据长度

- ✓ 返回值说明：

返回合封 EEPROM 操作的结果

typedef enum {

SIPEEPROM_RET_PASS = 0U, //操作成功

SIPEEPROM_RET_FAIL = 1U //操作失败

} SIPEEPROM_RET_TYPEDEF;

合封 EEPROM 写函数

- ✓ 函数说明：读取合封 EEPROM 的数据

- ✓ 函数原型：SIPEEPROM_RET_TYPEDEF LL_SIPEEPROM_Write(unsigned int addr, const unsigned char *data, unsigned int len)

- ✓ 参数说明：

unsigned int addr: 待写入合封 EEPROM 的地址

unsigned char *data: 待写入合封 EEPROM 的数据指针

unsigned len: 待写入数据长度

- ✓ 返回值说明：

返回合封 EEPROM 操作的结果

typedef enum {

SIPEEPROM_RET_PASS = 0U, //操作成功

SIPEEPROM_RET_FAIL = 1U //操作失败

} SIPEEPROM_RET_TYPEDEF;

合封 EEPROM 初始化

- ✓ 函数说明：合封 EEPROM 初始化函数，操作合封 EEPROM 前需调用该函数，注意调用该函数初始化合封 EEPROM 后，不能再操作合封 EEPROM 使用到的管脚，具体合封 EEPROM 使用到的管脚参见用户手册

- ✓ 函数原型：void LL_SIPEEPROM_Init(void);

- ✓ 参数说明：

无

- ✓ 返回值说明：

无

合封 EERPOM 复位函数

- ✓ 函数说明：复位合封 EEPROM
- ✓ 函数原型：void LL_SIPEEPROM_Rst(void)
- ✓ 参数说明：
无
- ✓ 返回值说明：
无

发生合封 EEPROM 读写错误，或系统进入休眠前，需调用复位合封 EEPROM 函数。

9 时钟调整函数

9.1 函数接口

函数接口：

eClkTrimRet_TypeDef LL_CLKTRIM_RchTrimByLosc(int *err_thousandth);
eClkTrimRet_TypeDef LL_CLKTRIM_RclTrimByRch(void);
eClkTrimRet_TypeDef LL_CLKTRIM_SysClkMonitorAsync(void);
eClkTrimRet_TypeDef LL_CLKTRIM_SysClkMonitorAsyncStatus(int *clk_jitter_ppm);
eClkTrimRet_TypeDef LL_CLKTRIM_SysClkMonitorSync(int *clk_jitter_ppm);
eClkTrimRet_TypeDef LL_CLKTRIM_LoscStopCheck(void);

9.2 函数说明

时钟调整函数仅适用于锐能微 V2/V3 版芯片。时钟调整函数不支持内置。

RCH 校准函数

- ✓ 函数说明：校准 RCH 时钟，校正基准来源于 LOSC
- ✓ 函数原型：eClkTrimRet_TypeDef LL_CLKTRIM_RchTrimByLosc(int *err_thousandth)
- ✓ 参数说明：
int *err_thousandth：调整后 RCH 的当前误差，单位为十万分之 1（RCH 的基准值为 29.4912MHz 或 32.768MHz，具体参见芯片用户手册）；该参数也可为空指针，当传入空指针时，函数不输出 RCH 精度误差值；

- ✓ 返回值说明：
返回 RCH 调整的结果

typedef enum {


```
CLK_TRIM_PASS = 0U,  
CLK_TRIM_TIMEOUT = 1U, //测量超时，建议重试  
CLK_TRIM_OUT_OF_RANGE = 2U, //RCH 超出范围，建议重试  
CLK_TRIM_UNSTABLE = 3U, //RCH 测量值不稳定，建议重试  
CLK_TRIM_NO_CLK = 4U, //RCH 测量过程中，未检测到有效时钟，建议排查软件是否  
开启 RCH 或 LOSC  
CLK_TRIM_REG_ERROR = 5U //RCH 测量过程中，寄存器读写错误，建议重试  
} eClkTrimRet_TypeDef;
```

- ✓ 应用说明 1：当系统使用内置 RCH 为主频时，为消除电压、温度等引起的时钟偏差，**要求用户必须定时调用该函数去调整芯片内部 RCH。定时周期建议选择 1 秒到 1 分钟之间。该函数执行时间在 700us 以上，故不可在中断中调用。**
- ✓ 应用说明 2：在没有 LOSC 的情况下，RCH 是无法校正的，校正函数会返回失败，RCH 的精度在 $\pm 1.5\%$ 左右。
- ✓ 应用说明 3：调用该函数后，不能再调用 RCH 校正参数校验库函数，否则 RCH 校准函数会不起作用。

RCL 校准函数

- ✓ 函数说明：采用芯片内部 RCH 校准 RCL 时钟函数，校准后 RCL 误差可控制在 0.4% 以内，仅 V2/V3 版芯片支持该函数
- ✓ 函数原型：eClkTrimRet_TypeDef LL_CLKTRIM_RclTrimByRch(void)
- ✓ 参数说明：无

- ✓ 返回值说明：

返回 RCL 调整的结果

```
typedef enum {
```

```
CLK_TRIM_PASS = 0U,  
CLK_TRIM_TIMEOUT = 1U, //测量超时，建议重试  
CLK_TRIM_OUT_OF_RANGE = 2U, //RCL 超出范围，建议重试  
CLK_TRIM_UNSTABLE = 3U, //RCL 测量值不稳定，建议重试  
CLK_TRIM_NO_CLK = 4U, //RCL 测量过程中，未检测到有效时钟，建议排查软件是否  
开启 RCH 或 LOSC  
CLK_TRIM_REG_ERROR = 5U //RCL 测量过程中，寄存器读写错误，建议重试  
} eClkTrimRet_TypeDef;
```

- ✓ 应用说明 1：在需要使用 RCL 作为 LCD 等时钟源的情况下，系统需要保证 RCL 的精度，此时可通过调用该函数保证 RCL 的精度；本函数单次调用时长约为 52ms，因此不建议客户频繁调用，可以定时（如 60s）调用一次，且不可在中断中调用。
- ✓ 应用说明 2：为了安全，每调用一次该函数，只能校正一个 step；负方向-0.49%每 step，正方向+0.45%每 step。
- ✓ 应用说明 3：调用该函数后，不能再调用 RCL 校正参数校验库函数，否则 RCL 校准函数会不起作用

系统时钟同步监控函数

- ✓ 函数说明：系统时钟同步监控函数，该函数可用于 PLL 作为系统主频时，使用内部 RCH 监控 PLL 时钟的稳定性。
- ✓ 函数原型：eClkTrimRet_TypeDef LL_CLKTRIM_SysClkMonitorSync(int *clk_jitter_ppm)
- ✓ 参数说明：芯片内 PLL 相对 RCH 时钟的相对偏差，单位为 ppm；如果该偏差稳定（如偏差跳动小于 2000ppm），则认为时钟 PLL 和 RCH 稳定。

- ✓ 返回值说明：

返回系统时钟时钟监控的结果

```
typedef enum {  
    CLK_TRIM_PASS = 0U,  
    CLK_TRIM_TIMEOUT = 1U, //测量超时，建议重试  
    CLK_TRIM_OUT_OF_RANGE = 2U, //时钟超出范围，建议重试  
    CLK_TRIM_UNSTABLE = 3U, //时钟测量值不稳定，建议重试  
    CLK_TRIM_NO_CLK = 4U, //时钟测量过程中，未检测到有效时钟，建议排查软件是否  
    开启 RCH 或 LOSC  
    CLK_TRIM_REG_ERROR = 5U //时钟测量过程中，寄存器读写错误，建议重试  
} eClkTrimRet_TypeDef;
```

- ✓ 应用说明：该函数为阻塞型，调用后，函数内部启用芯片的内的时钟测量功能，以 PLL 为基准时钟，RCH 为被测时钟，输出 RCH 相对 PLL 的一个偏差量。该过程需要耗时约 1ms，建议主函数内定时调用，**定时周期建议选择 1 秒到 1 分钟之间。**
- ✓ 应用场景：
需要监控系统时钟稳定性的场合。
- ✓ 适用范围：该函数仅适用于 V2/V3 版芯片

系统时钟异步监控函数

- ✓ 函数说明：系统时钟异步监控函数，该函数可用于 PLL 作为系统主频时，使用内部 RCH 监控 PLL 时钟的稳定性
- ✓ 函数原型：eClkTrimRet_TypeDef LL_CLKTRIM_SysClkMonitorAsync(void)
- ✓ 参数说明：无

- ✓ 返回值说明：

返回系统时钟监控的结果

```
typedef enum {  
    CLK_TRIM_PASS = 0U,  
    CLK_TRIM_TIMEOUT = 1U, //测量超时，建议重试  
    CLK_TRIM_OUT_OF_RANGE = 2U, //时钟超出范围，建议重试  
    CLK_TRIM_UNSTABLE = 3U, //时钟测量值不稳定，建议重试  
    CLK_TRIM_NO_CLK = 4U, //时钟测量过程中，未检测到有效时钟，建议排查软件是否  
    开启 RCH 或 LOSC
```

- CLK_TRIM_REG_ERROR = 5U //时钟测量过程中，寄存器读写错误，建议重试
- } eClkTrimRet_TypeDef;
- ✓ 应用说明：调用该函数后，会启动芯片的内的时钟测量功能，以 PLL 为基准时钟，RCH 为被测时钟，进行 RCH 相对 PLL 的一个相对偏差量测试。测试耗时约 35ms，测试完成后，可通过“系统时钟异步监控结果获取函数”获取偏差量；推荐调用该函数时，调用 NVIC_EnableIRQ(SYSCLKCAL_IRQn)开启系统时钟校正中断，在中断处理函数 SYSCLKCAL_HANDLER 中调用“系统时钟异步监控结果获取函数 LL_CLKTRIM_SysClkMonitorAsyncStatus”获取时钟监控结果。系统时钟异步监控结果获取函数内有清中断标志操作，无需单独处理。
 - ✓ 应用场景：需要进行非阻塞方式时钟准确性监控的场景下。
 - ✓ 适用范围：该函数仅适用于 V2/V3 版芯片

系统时钟异步监控结果获取函数

- ✓ 函数说明：系统时钟监控函数
- ✓ 函数原型：eClkTrimRet_TypeDef LL_CLKTRIM_SysClkMonitorAsyncStatus(int *clk_jitter_ppm)
- ✓ 参数说明：无

- ✓ 返回值说明：

返回 RCL 调整的结果

typedef enum {

CLK_TRIM_PASS = 0U,

CLK_TRIM_TIMEOUT = 1U, //测量超时，建议重试

CLK_TRIM_OUT_OF_RANGE = 2U, //时钟超出范围，建议重试

CLK_TRIM_UNSTABLE = 3U, //时钟测量值不稳定，建议重试

CLK_TRIM_NO_CLK = 4U, //时钟测量过程中，未检测到有效时钟，建议排查软件是否开启 RCH 或 LOSC

CLK_TRIM_REG_ERROR = 5U //时钟测量过程中，寄存器读写错误，建议重试

} eClkTrimRet_TypeDef;

- ✓ 适用范围：该函数仅适用于 V2/V3 版芯片

LOSC 停振检测函数

- ✓ 函数说明：LOSC 停振检测函数，当 LOSC 异常时，会返回错误状态
- ✓ 函数原型：eClkTrimRet_TypeDef LL_CLKTRIM_LoscStopCheck(void)
- ✓ 参数说明：无

- ✓ 返回值说明：

返回 LOSC 的状态检测结果

```
typedef enum {  
    CLK_TRIM_PASS = 0U,  
    CLK_TRIM_TIMEOUT = 1U, //测量超时，建议重试  
    CLK_TRIM_OUT_OF_RANGE = 2U, //时钟超出范围，建议重试  
    CLK_TRIM_UNSTABLE = 3U, //时钟测量值不稳定，建议重试  
    CLK_TRIM_NO_CLK = 4U, //时钟测量过程中，未检测到有效时钟，建议排查软件是否  
    开启 RCH 或 LOSC  
    CLK_TRIM_REG_ERROR = 5U //时钟测量过程中，寄存器读写错误，建议重试  
} eClkTrimRet_TypeDef;
```

- ✓ 应用说明 1: 可通过调用该函数检测 LOSC 是否停振；当该函数检测到 LOSC 的状态异常时，可通过多次调用该函数以确认为真实停振。
- ✓ 应用说明 2: 该函数检测的对象是 LL_RTC_ClkSel 配置的时钟，默认为 LOSC

10 GPADC 操作函数

10.1 函数接口

函数接口：

```
eGpadcLibRet_TypeDef LL_GPADC_TadjGet(unsigned int *tadj);
```

10.2 函数说明

GPADC 温度偏置获取函数

- ✓ 函数说明：获取三相计量 SOC V2 版芯片的 GPADC 温度偏置值
- ✓ 函数原型：eGpadcLibRet_TypeDef LL_GPADC_TadjGet(unsigned int *tadj)
- ✓ 参数说明：
 unsigned int *tadj: GPADC 温度偏置值

- ✓ 返回值说明：

返回 RCH 调整的结果

```
typedef enum  
{  
    GPADC_LIB_PASS = 0, //获取 GPADC 温度偏置值成功  
    GPADC_LIB_FAIL = 1, //获取 GPADC 温度偏置值失败  
} eGpadcLibRet_TypeDef;
```

- ✓ 适用范围：仅适用于三相计量 SOC 版芯片。

11 计量操作函数

11.1 函数接口

函数接口：

<code>eEmuLibRet_TypeDef LL_EMU_ExtRefConf(eExtRefSel_TypeDef ref_sel);</code>
--

11.2 函数说明

外部基准控制函数

- ✓ 函数说明：使能或禁止三相 SOC V2 版芯片计量外灌基准
- ✓ 函数原型：`eEmuLibRet_TypeDef LL_EMU_ExtRefConf(eExtRefSel_TypeDef ref_sel)`
- ✓ 参数说明：

`eExtRefSel_TypeDef ref_sel`：外部基准控制参数

typedef enum

```
{  
    EXTREF_ENABLE = 0, //使能外部基准  
    EXTREF_DISABLE = 1 //禁止外部基准  
} eExtRefSel_TypeDef;
```

- ✓ 返回值说明：

返回 RCH 调整的结果

typedef enum

```
{  
    EMU_LIB_PASS = 0, //函数操作成功  
    EMU_LIB_FAIL = 1 //函数操作失败  
} eEmuLibRet_TypeDef;
```

- ✓ 适用范围：仅适用于三相计量 SOC 版芯片。

12 内置库校验函数

12.1 函数接口

函数接口：

```
NvrLibValid_TypeDef LL_LIB_Check(void);
```

12.2 函数说明

内置库校验函数

- ✓ 函数说明：验证芯片内置库函数的有效性
 - ✓ 函数原型：NvrLibValid_TypeDef LL_LIB_Check(void)
 - ✓ 参数说明：
无

 - ✓ 返回值说明：
库函数是否有效
- ```
typedef enum {
 IS_LIB_VALID = 0U, //内置库函数有效
 IS_LIB_INVALID = 1U //内置库函数无效
} NvrLibValid_TypeDef;
```
- ✓ 应用说明：如果该函数返回 **IS\_LIB\_INVALID**，请勿调用内置库函数，可做芯片异常事件处理。